# INF221 Assignment 1

| Identification | |
|---|---|
| Group Number | 1 |
| Members | • Dan Trung Nguyen<br>• Håkon Bråthen Børve<br>• Milad Tootoonchi |

## Exercise 1

Note: This code uses 0-based code, meaning the first element in the array has the index 0.

In [ ]:
```
A = [8, 9, 2, 3, 7, 1]
temp_A = A.copy()

n = len(A)
for j in range(1, n):
    key = temp_A[j]     # Select current number
    i = j - 1           # Select previous index

    print(j, i, key, temp_A)          # Print out values for the table

    while i > -1 and temp_A[i] > key: # Goes "down" the array and checks if the pre
        temp_A[i + 1] = temp_A[i]
        i = i - 1

    temp_A[i + 1] = key


print(temp_A)
```

```
1 0 9 [8, 9, 2, 3, 7, 1]
2 1 2 [8, 9, 2, 3, 7, 1]
3 2 3 [2, 8, 9, 3, 7, 1]
4 3 7 [2, 3, 8, 9, 7, 1]
5 4 1 [2, 3, 7, 8, 9, 1]
[1, 2, 3, 7, 8, 9]
```

Illustration of pseudocode:
Total loops: 6. We've already printed the table above, but this assumes a 0-based indexing.
We will therefore add 1 to every **i** and **j** in order to make our table into an 1-based indexing.

| j | i | key | A |
|---|---|---|---|
| 2 | 1 | 9 | [8, 9, 2, 3, 7, 1] |

| | | | |
|---|---|---|---|
| 3 | 2 | 2 | [8, 9, 2, 3, 7, 1] |
| 4 | 3 | 3 | [2, 8, 9, 3, 7, 1] |
| 5 | 4 | 7 | [2, 3, 8, 9, 7, 1] |
| 6 | 5 | 1 | [2, 3, 7, 8, 9, 1] |
| Terminated | | | [1, 2, 3, 7, 8, 9] |

# Exercise 2

Linear search is a search algorithm where we go through every element in a list. Starting from the first one, and then continuing through the list until we find our desired element. This can be compared to a for-loop with an if statement. Here is the pseudocode:

**Input:** A sequence of $n$ numbers $A =< a_1. a_2, \ldots, a_n >$ and a value $v$
**Output:** An index $i$ such that $v = A[i]$ or the special value NIL if does not appear in $A$

This pseudocode will use a 1-based indexing system as most do.

```
FUNCTION LinearSearch(List, Target)
    for i = 1 to List.length
        if List[i] == Target
            return i

    return NIL

LinearSearch(A, v)
```

If the for-loop does not find $v$, then it'll run the last line, returning $NIL$.

The runtime for this algorithm with an array of size $n$ would be:

- Best case scenario
  $O(1)$ - Speed is always constant, as in the best case scenario the algorithm finds the target in the first ever loop.
- Average case $O(n)$ - The average runtime is $\frac{n}{2}$, meaning the average runtime is $O(\frac{n}{2})$. However, both are still linear, and since we want the least complex version of $O$, we convert $O(\frac{n}{2})$ to $O(n)$.
- Worst case scenario $O(n)$ - Similar runtime to the average runtime. In this scenario, the target is placed at the very last spot in the array. This is still $O(n)$ because the algorithm scales based on $n$ linearly. In other words, an array of $n$ will take $n$ loops to find the target, while an array of $n + 1$ will find the target in $n + 1$ loops.

# Exercise 3

1. Names in a phonebook are a nominal feature, meaning that it doesn't make sense to order them in any way. By converting the strings into keys, we are able to.
2. The converted number/keys from a string must be unique such that there is no mapping many to one (or if one intends to be able to map back to the original strings) or one to many.
3. The algorithm will take each character of the strings and convert into 8-bits via utf-8 thus for a 8-byte string we will have a corrosponding 8-byte number. this can have a potential problem if one-to-one mapping is desired in case of identical strings.

# Exercise 4

One of the best qualities of insertion sort is that it is very efficient with smaller array sizes. Since the question asks about "which values" we are safe to assume that there are multiple values where insertion sort is worse than merge sort. We're not sure about which $\lg$ is used, so we're calculating for both $\log_2$ and $\log_{10}$.

In [34]:
```
import numpy as np

# Define runtime functions
def insert_sort_runtime(n):
    return 16 * n ** 2

def merge_sort_runtime_log10(n):
    return 512 * n * np.log10(n)

def merge_sort_runtime_log2(n):
    return 512 * n * np.log2(n)

MergeBetter = False
n = 2 # Start at array size 2, anything is sorted at n = 1

# log_10
# Run until we find an array size where merge sort is better
while not MergeBetter:
    if insert_sort_runtime(n) < merge_sort_runtime_log10(n): # Compare values
        n += 1
    else:
        print(f"Merge sort: {merge_sort_runtime_log10(n)} (log_10)\nInsertion sort:
        MergeBetter = True

# log_2
MergeBetter = False
n = 2
while not MergeBetter:
    if insert_sort_runtime(n) < merge_sort_runtime_log2(n): # Compare values
        n += 1
    else:
```

```
        print(f"Merge sort: {merge_sort_runtime_log2(n)} (log_2)\nInsertion sort: {
        MergeBetter = True
```

```
Merge sort: 50124.04711032178 (log_10)
Insertion sort: 50176
n = 56


Merge sort: 1048576.0 (log_2)
Insertion sort: 1048576
n = 256
```

We see that merge sort and insert sort are equal at $n = 256$. Therefore Merge > Insert at $n = 255$.

Insertion sort is faster than merge sort in:

- $n \in [2, 255]$ if $\log_2$
- $n \in [2, 55]$ if $\log_{10}$

# Exercise 5

Saying that the run time of an algorithm is <u>at least</u> $O(n^2)$ is meaningless because of the lack of information. The $O$ notation is used to provide an upper bound for the algorithm, while the statement is using $O$ as a lower bound.

In short, you cannot set an upper bound $O(n^2)$ as a lower bound, as it's a contradiction. The correct usage of this would be saying that the run time of the algorithm is at most $O(n^2)$. Use $\Omega(n^2)$ if you want to describe lower bound.

# Exercise 6

$$T(n) = 2n^3 - 5\sqrt{n} + 4 \lg n$$

$2n^3$ is the dominant term since the rest of the equation increase slower.
There exist contants $c_1, c_2, n_0$ such that for all $n \geq n_0$:

$$c_1 n^3 \leq T(n) \leq c_2 n^3$$

**Lower Bound**
since:

$$\sqrt{n} = o(n^3)$$

then eventually:

$$5\sqrt{n} \leq n^3$$

so for large n:

$$T(n) \geq 2n^3 - n^3 = n^3$$

$$=> T(n) \geq n^3$$

and therfore we choose $c_1 = 1$

**Upper Bound** For large n:

- $$\lg n \leq n^3$$
- $$-5\sqrt{n} \leq 0$$

so:

$$T(n) = 2n^3 - 5\sqrt{n} + 4\lg n$$

The upper bound should be larger then T(n) and therefore we can remove $-5\sqrt{n}$, since $0 \geq -5\sqrt{n}$

$$=> T(n) \leq 2n^3 - 5\sqrt{n} + 4n^3 \leq 6n^3$$

and therefore we choose $c_2 = 6$

Thus, there exist constants $c_1 = 1$ and $c_1 = 6$ and a large $n_0$ such that:

$$n^3 \leq T(n) \leq 6n^3$$

for all $n \leq n_0$ therefore:

$$T(n) \in \Theta(n^3)$$

# Exercise 7

## 1.

In [25]:
```python
def recursive_min(arr, n=None):
    if n is None:
        n = len(arr)
    if n == 1:
        return arr[0]
    prev_min = recursive_min(arr, n - 1)
    return prev_min if prev_min < arr[n - 1] else arr[n - 1]
```

## 2.

It is not a branching tree, just a single chain:
T(n) -> T(n-1) -> T(n-2) -> ... ->T(1)

At each level it is called one recursive, one comparison and constant work.
There are n levels so total work:

$$T(n) = n * O(1) = O(n)$$

## 3.

**Recurrence Equation**

$$T(n) = T(n-1) + c$$

with T(1) = c:

$$=> T(n) = T(1) + (n-1)c => T(n) = nc$$

$$T(n) \in \Theta(n)$$

## 4.

In [26]:
```
# From the lab week 6
def minimum(data):
    min_value = data[0]
    for next_value in data[1:]:
        if next_value < min_value:
            min_value = next_value
    return min_value
```

In [27]:
```
import random
# creating test data
l = [random.randint(0, 100000) for _ in range(500)]

%timeit recursive_min(l)
%timeit minimum(l)
```

```
71.7 µs ± 296 ns per loop (mean ± std. dev. of 7 runs, 10,000 loops each)
9.79 µs ± 636 ns per loop (mean ± std. dev. of 7 runs, 100,000 loops each)
```